



Practica 3

Explicacion Practica 3

Practica3_2026.pdf

Ejercicio 1: ¿Qué define la semántica?

-Describe el significado de simbolos y palabras de un lenguaje natural o informatico. Este lenguaje tiene que ser sintacticamente valido

Ejercicio 2:

1. ¿Qué significa compilar un programa?

-Compilar un programa significa traducir el codigo fuente escrito en lenguaje de alto nivel a un lenguaje de computadora para que esta misma puede entenderlo, es decir, a lenguaje de maquina. La compilacion se realiza por un compilador el cual analiza todo el codigo fuente antes de ejecutarlo y genera un archivo ejecutable

2. Describa brevemente cada uno de los pasos necesarios para compilar un programa.

- 1) Etapa de analisis

- Analisis lexico (Scanner): hace el analisis a nivel de palabra (lexema) y divide al programa en categorias. Estas categorias pueden ser identificadores, delimitadores, simbolos especiales, numeros, etc y analiza el tipo de cada uno para ver si son TOKENS validos. Por ultimo filtra comentarios y separadores (espacios en blanco, tabulaciones). LLeva una tabla para la especificacion del analizador lexico que incluye cada categoria, atributos y acciones asociadas. Pone identificadore en la tabla de simbolos y reemplaza cada simbolo por su entrada en la tabla y genera errores si esa entrada no coincide con ninguna categoria

lexica. El resultado de este paso sera el descubrimiento de tokens y deteccion de errores

-Analisis sintactico (Parser): hace el analisis a nivel de sentencia y estructuras. Identifica estructura de las sentencias, expresiones, declaraciones que estan en su entrada utilizando los tokens previos del analizador lexico. Las estructuras pueden ser representadas con arboles sintacticos/derivacion los cuales explican como se deriva la cadena de entrada en la gramatica del lenguaje con el objetivo de validar que lo que entra es correcto. El parser y el scanner se alternan

-Analisis semantico (Semantica estatica): hace el analisis luego de que el scanner y el parser realizan su analisis correctamente. Se encarga de procesar estructuras sintacticas comprobando los tipos (gramatica de atributos), agregando a la tabla de simbolos los descriptores de tipos, realizando comprobaciones de duplicados/nombres/tipos, etc.

- **1.5) Generacion de codigo intermedio**: ocurre entre la etapa de analisis y de sintesis y es una transformacion de codigo fuente en una representacion de codigo intermedio para una maquina que es abstracta. Esta representacion es independiente de la maquina que ejecuta el programa y presenta un codigo de 3 direcciones

- 2) Etapa de sintesis

-Optimizacion de codigo: Es opcional y no todos los compiladores pueden hacerlo tampoco. Hay diversas formas de hacer optimizacion como por ejemplo preferir velocidad de ejecucion y tamaño de codigo ejecutable, evaluacion de expresiones booleanas, eliminacion de codigo muerto/funciones que no se utilizan

-Generacion de codigo final: Se traduce el codigo fuente por medio del compilador a codigo de maquina o codigo intermedio que es el que puede ejecutar la computadora

3. ¿En qué paso interviene la semántica y cual es su importancia dentro de la compilación?

-Interviene en 1) Etapa de analisis. Es una de las partes mas importantes ya que es el nexo entre las dos etapas y esta previa a la generacion de codigo intermedio. Se encarga de encontrar los ultimos errores que se pueden haber pasado por alto por los primeros 2 analisis y analiza estructuras sintacticas

Ejercicio 3: Con respecto al punto anterior ¿es lo mismo compilar un programa que interpretarlo? Justifique su respuesta mostrando las diferencias básicas, ventajas y desventajas de cada uno.

▼ COMPARACION CATEGORIA A CATEGORIA DE INTERPRETE Y COMPILADOR

- **Por cómo se ejecuta:**

- **Intérprete:**

- Se utiliza el intérprete en la ejecución
 - Ejecuta el programa línea por línea
 - Por donde pase **dependerá** de la **acción** del **usuario**, de la **entrada de datos** y/o de alguna **decisión del programa**
 - Siempre se debe **tener el Programa Interprete**
 - El **programa fuente** será **público** (necesito ambos)

- **Compilador:**

- Se utiliza el compilador **antes de la ejecución**
 - Produce un **programa ejecutable** equivalente en **lenguaje** a **objeto**
 - El **programa fuente** no será **público**

Comparación entre Compilador e Intérprete (cont..)

- **Por el orden de ejecución:**

- **Intérprete:**

- Sigue el **orden lógico** de ejecución (**no necesariamente recorre todo el código**)

- **Compilador:**

- Sigue el **orden físico** de las sentencias (**recorre todo**)

Comparación entre Compilador e Intérprete (cont..)

○ Por el tiempo de ejecución del:

Intérprete:

- Por cada sentencia que pasa realiza el proceso de **decodificación (lee, analiza y ejecuta)** para determinar las operaciones y sus operandos. **Es repetitivo**
- Si la sentencia está en un **proceso iterativo** (ej.: for/while), se realizará la tarea de decodificación **tantas veces como sea requerido**
- La **velocidad** de proceso se puede ver **afectada** por esto

Compilador:

- Pasa por **todas las sentencias**.
- **No repite lazos**
- **Traduce todo de una sola vez.**
- Genera **código objeto ya depurado y compilado.**
- La **velocidad de compilar** dependerá del **tamaño del código**

57

Comparación entre Compilador e Intérprete (cont..)

○ Por la eficiencia posterior:

Intérprete:

- **Más lento** en ejecución. Se **repite el proceso** cada vez que se **ejecuta el mismo programa** o **pasa por las mismas instrucciones**
- **Para ser ejecutado en otra máquina** se necesita tener si o si el **intérprete instalado** y el **programa fuente** será **público**

Compilador:

- **Más rápido** ejecutar desde el **punto de vista del hardware** porque ya **está en un lenguaje de más bajo nivel** y **listo para ser ejecutado.**
- Ya **compilado** es **más eficiente.**
- De acuerdo al código tal vez se **tardó más en compilar** porque se **verificó y detectó** todo previamente.

58

Comparación entre Compilador e Intérprete (cont..)

- **Por el espacio ocupado - Intérprete:**
- El programa **no se traduce completamente** antes de ejecutarse. Se **interpreta instrucción por instrucción** según la ejecución.
- La memoria se ocupa con:
 - El **programa fuente**, se mantiene en su forma original (**texto de programa, árbol sintáctico**)
 - La **tabla de símbolos, variables y otras estructuras** que se generen cuando se usan en forma **dinámica**
 - El **intérprete** contiene los **subprogramas** necesarios para ejecutar todas las instrucciones (**asignación, if, bocle, etc.**)
 - El **intérprete**. Puede **cargarse una sola vez** y luego ejecutar **distintos programas** (ej. **Python y Ruby**)
- **Importante:** El espacio ocupado depende del tamaño del programa y del intérprete. Si el programa es muy pequeño, el intérprete completo puede ocupar más memoria que un ejecutable compilado

59

Comparación entre Compilador e Intérprete (cont..)

- **Por el espacio ocupado - Compilador**
- El programa se **traduce completamente** antes de ejecutarse, generando un **programa ejecutable**.
- La memoria se utiliza para:
 - El **programa compilado (código máquina)**. Recordar que **varias instrucciones de máquina** pueden generarse a partir de una sola instrucción de alto nivel
 - **Datos del programa** durante la ejecución
 - **Bibliotecas necesarias** para la ejecución
 - El **compilador no necesita estar en memoria durante la ejecución**, solo el programa ejecutable.
- **Ventaja:** La ejecución suele ser **más rápida**, programa ya traducido.
- **Importante:** El espacio ocupado depende del tamaño del programa y de la implementación del lenguaje, por lo que un programa compilado no siempre ocupa más memoria que uno interpretado.

60

Comparación entre Compilador e Intérprete (cont..)

- Por el espacio ocupado
 - No es correcto afirmar simplemente:
 “el intérprete ocupa menos memoria”
 “el compilado ocupa más memoria”
 - En realidad depende de dos factores:
 - El tamaño del programa que se desea ejecutar
 - La forma en que el lenguaje está implementado (compilador o intérprete)
- Porque:
- en **interpretación** hay que cargar el **intérprete completo**
 - en **compilación** si el programa es grande, el **ejecutable también puede ser grande**

61

Comparación entre Compilador e Intérprete (cont..)

- Por la detección de errores:
 - Intérprete:
 - Las **sentencias del código fuente** pueden ser **relacionadas directamente con la sentencia en ejecución**, entonces se puede ubicar donde se produjo el error.
 - Es más fácil detectarlos por donde pasa la ejecución
 - Es más fácil corregirlos
 - Compilador:
 - Se pierde la **referencia** entre el **código fuente** y el **código objeto**.
 - Es **casi imposible** ubicar el error
 - Se deben usar otras técnicas (ej. Semántica Dinámica)

62

- Diferencias básicas:

- Interpretación:

- La traducción se realiza en tiempo de ejecución.
- El intérprete procesa el programa línea por línea (Leer → Analizar → Ejecutar).

- No genera un archivo ejecutable permanente.
- Solo se ejecutan las instrucciones necesarias según el flujo del programa.

-Compilacion:

- La traducción se realiza antes de la ejecución.
- El compilador traduce todo el programa completo a lenguaje máquina.
- Genera un archivo ejecutable (código objeto).
- El programa ya compilado puede ejecutarse varias veces sin recompilar.

-Ventajas y desventajas:

-Interpretacion:

Ventajas:

- Permite detectar errores rápidamente, ya que se ejecuta paso a paso.
- Facilita la depuración (debugging).
- Es más flexible y portable (mismo código en distintas plataformas con el intérprete adecuado).

Desventajas:

- Menor rendimiento, porque traduce y ejecuta al mismo tiempo.
- Requiere el intérprete cada vez que se ejecuta.
- Puede ser menos eficiente en programas grandes.

-Compilacion:

Ventajas:

- Mayor velocidad de ejecución, ya que el programa ya está traducido.
- No necesita el compilador para ejecutarse (solo el ejecutable).
- Mejor rendimiento en aplicaciones grandes o críticas.

Desventajas:

- Tiempo de compilación previo antes de probar el programa.
- Los errores se detectan después de compilar todo el código.
- Menor flexibilidad frente a cambios pequeños (hay que recompilar).

Ejercicio 4: Explique claramente la diferencia entre un error sintáctico y uno semántico. Ejemplifique cada caso.

-Error sintactico: ocurre cuando el parser detecta que un código no sigue las reglas gramaticales del lenguaje de programación

Ejemplo en Pascal: $x = 3 \rightarrow$ Pascal usa $:=$ para asignación

-Error semantico: ocurre cuando un código sintácticamente correcto pero no produce un resultado esperado debido a que se interpreta incorrectamente el significado del código

Ejemplo en Java:

String x;

$x = 3 \rightarrow$ Se intenta asignar un integer a un String

Ejercicio 5: Sean los siguientes ejemplos de programas. Analice y diga qué tipo de error se produce (Semántico o Sintáctico) y en qué momento se detectan dichos errores (Compilación o Ejecución). Aclaración: Los valores de la ayuda pueden ser mayores.

```
a) Pascal
Program P --> Falta ; (1er error sintactico, compilacion)
var 5: integer; --> Nombre de variable con numeros (2do error sintactico, compilacion)
var a: char;
Begin
  for i:=5 to 10 do begin --> Variable i no declarada (1er error semantico, compilacion)
    write(a); --> Variable a no esta inicializada (2do error semantico, ejecucion)
    a= a + 1
      --> Usa "=" en vez de ":=" (3er error sintactico, compilacion)
      --> a es de tipo char y se le suma un int (3er error semantico, compilacion)
```



```
end;  
End.
```

Ayuda: Sintáctico 2, Semántico 3

b) Java

```
public String tabla(int numero, arrayList<Boolean> listado)  
{  
    --> es ArrayList (1er error sintactico, compilacion)  
    String result = null;  
    for(i = 1; i < 11; i--) {  
        --> i no tiene tipo y ademas es bucle infinito (2do error sintactico 1er error logico, compilacion)  
        result += numero + "x" + i + "=" + (i*numero) + " n";  
        --> concatena null con string (1er error semantico, ejecucion)  
        listado.get(listado.size()-1)=(BOOLEAN) numero>i;  
        --> BOOLEAN es incorrecto (2do error semantico, compilacion)  
        --> Asignacion invalida (3er error sintactico, compilacion)  
        --> Estan mal los parentesis (4to error sintactico, compilacion)  
    }  
    return true; --> Retorna true y deberia ser String (3er error semantico, compilacion)  
}
```

Ayuda: Sintácticos 4, Semánticos 3, Lógico 1

c) C

```
# include <stdio.h>  
int suma; /* Esta es una variable global */  
int main() {  
    int indice;  
    encabezado; --> No esta definido previamente (1er error semantico, compilacion)
```

```

    for (indice = 1 ; indice <= 7 ; indice ++)  

        --> indice esta con espacio (1er error sintacti  
co)  

        cuadrado (indice);--> No esta declarado (2do error sema  
ntico, compilacion)  

        final(); Llama a la función final */  

        -> No esta declarado (3er error semantico, compilac  
ion)  

        -> Comentario no abierto (1er error sintactico, com  
pilacion)  

        return 0;  

    }  

    cuadrado (numero)  

        -> Numero esta declarado despues (4to error semantico,  
compilacion)  

        -> No se abren llaves para la funcion (2do error sintac  
tico, compilacion)  

    int numero;  

    { int numero_cuadrado;  

        numero_cuadrado == numero * numero;  

        --> Numero no esta inicializada (5to error semantic  
o, compilacion)  

        --> Usa == en vez de = (3er error sintactico, compila  
cion)  

        suma += numero_cuadrado;  

        -> suma no esta inicializa (6to error semantico, co  
mpilacion)  

        printf("El cuadrado de %d es %d\n",  

            numero, numero_cuadrado);  

    }

```

Ayuda: Sintácticos 2, Semánticos 6

d) Python

```
#!/usr/bin/python
```

```
print "\nDEFINICION DE NUMEROS PRIMOS"
```

```

-> Faltan parentesis (1er error sintactico, compilacio
n)
r = 1
while r = True:--> Es == (2do error sintactico, compilacio
n)
    N = input("\nDame el numero a analizar: ")
    -> No castea a int (1er error semantico, compilacio
n)
    i = 3
    fact = 0
    if (N mod 2 == 0) and (N != 2):-> Es % no mod (3er erro
r sintactico, compilacion)
        print "\nEl numero %d NO es primo\n" % N
        -> Faltan parentesis (4to error sintactico, com
pilacion)
    else:
        while i <= (N^0.5):-> Se usa ** no ^, (5to error si
ntactico, compilacion)
            if (N % i)== 0:
                mensaje="\nEl numero ingresado NO es primo
\n" % N
                -> Faltan parentesis (6to error sintact
ico, compilacion)
                -> Esta raro el % N (2do error semantic
o, compilacion)
                msg = mensaje[4:6]
                print msg
                -> Faltan parentesis (7mo error sintact
ico, compilacion)
                fact = 1
                i+=2
            if fact == 0:
                print "\nEl numero %d SI es primo\n" % N
        r = input("Consultar otro número? SI (1) o NO (0)--->>
")

```

Ayuda: Sintácticos 2, Semánticos 3

e) Ruby

```
def ej1
```

```
Puts 'Hola,¿Cuál es tu nombre?'-> Es puts (1er error sintac  
tico, compilacion)
```

```
nom = gets.chomp
```

```
puts 'Mi nombre es ', + nom
```

```
-> La , esta mal puesta para concatenar (2do error sint  
actico, compilacion)
```

```
puts 'Mi sobrenombre es 'Juan''
```

```
-> Comillas simples dentro de simples (3er error sintac  
tico, compilacion)
```

```
puts 'Tengo 10 años'
```

```
meses = edad*12
```

```
-> edad no esta definida (1er error semantico, ejecucio  
n (ruby))
```

```
dias = 'meses' *30
```

```
-> Multiplica string 30 veces (2do error semantico, eje  
cucion)
```

```
hs= 'dias * 24'
```

```
puts 'Eso es: meses + ' meses o ' + dias + ' días o ' + hs  
+ ' horas'
```

```
-> El + queda metido en la cadena y no concatena lo que  
sigue (3er error semantico, 4to error sintactico, ejecucio  
n)
```

```
puts 'vos cuántos años tenés'
```

```
edad2 = gets.chomp
```

```
edad = edad + edad2.to_i
```

```
puts 'entre ambos tenemos ' + edad + ' años'
```

```
puts '¿Sabes que hay ' + name.length.to_s + ' caracteres en  
tu nombre, ' + name + '?'
```

```
-> name no esta definida (4to error semantico, ejecucio  
n)
```

```
end
```

Ayuda: Semánticos +4

Ejercicio 5: Dado el siguiente código escrito en pascal. Transcriba la misma funcionalidad de acuerdo al lenguaje que haya cursado en años anteriores. Defina brevemente la sintaxis (sin hacer la gramática) y semántica para la utilización de arreglos y estructuras de control del ejemplo.

```
Procedure ordenar_arreglo(var arreglo: arreglo_de_caract
eres;cont:integer);
var
i:integer; ordenado:boolean;
aux:char;
begin
    repeat
        ordenado:=true;
        for i:=1 to cont-1 do
            if ord(arreglo[i])>ord(arreglo[i+1])
                then begin
                    aux:=arreglo[i];
                    arreglo[i]:=arreglo[i+1];
                    arreglo[i+1]:=aux; ordenado:=false
                end;
        until ordenado;
    end;
```

Observación: Aquí sólo se debe definir la instrucción y qué es lo que hace cada una; detallando alguna particularidad del lenguaje respecto de ella. Por ejemplo el for de java necesita definir una variable entera, una condición y un incremento para dicha variable.

```
## Paso el codigo a python, no requiero declaracion de v
ariables ya que es dinamico
## el recibimiento de parametros no requiere especificac
ion de tipos
## requiere identacion y uso de ":" en funciones y estru
cturas
## el for de Python necesita un rango, salgo del bucle c
on un break en vez
## de hacer una iteracion mas con el repeat until
```

```
def ordenar_arreglo(arreglo, cont):
    while True:
        ordenado = True
        for i in range(cont - 1):
            if ord(arreglo[i]) > ord(arreglo[i + 1]):
                aux = arreglo[i]
                arreglo[i] = arreglo[i + 1]
                arreglo[i + 1] = aux
                ordenado = False
        if ordenado:
            break
```

Ejercicio 6: Explique cuál es la semántica para las variables predefinidas en lenguaje Ruby self y nil. ¿Qué valor toman; cómo son usadas por el lenguaje?

-self:

- Es una variable que hace referencia al objeto actual (this de java)
- En un metodo de instancia hace referencia a la instancia del objeto
- En un metodo de clase o en un clase hace referencia a la clase. Y si esta en el contexto de un modulo hace referencia al modulo
- Fuera de cualquier clase o modulo self hace referencia al objeto principal y al inicio del interprete tiene el valor main (primer objeto creado)

En resumen, se usa para acceder a metodos y propiedades del objeto o la clase actual y al definir los metodos de clase y de instancia

-nil:

- Representa la ausencia de valor/referencia vacia, por lo que es un objeto especial que se utiliza para indicar valores no asignados. Tambien puede ser usado como falta de datos o para indicar el valor de retorno default de un metodo

Ejercicio 7: Determine la semántica de null y undefined para valores en javascript. ¿Qué diferencia hay entre ellos?

- null: Se usa para indicar que una variable no tiene valor y que fue establecida específicamente para que no tenga valor, es decir, una variable

no puede ser null a menos que explícitamente este establecida como null en algun lado

-undefined: Significa que no hay valor en una variable porque no se le ha asignado ninguno. El momento en el que una variable este declarada pero no inicializada es cuando toma el valor de undefined. Se puede setear manualmente para reestablecer una variable

En resumen null es primitivo y se setea manualmente en variables/retornos y undefined es el valor predeterminado que toma una variable/retornos aunque AMBOS indican la ausencia de valor. La diferencia esta es que null es seteado explícitamente para indicar que algo no tiene valor y undefined indica que algo no tiene valor momentaneamente ya que todavia no se le ha asignado o cuando se accede a una propiedad inexistente del objeto

Ejercicio 8: Determine la semántica de la sentencia break en C, PHP, javascript y Ruby. Cite las características más importantes de esta sentencia para cada lenguaje

Lenguaje	Semántica de <code>break</code>	Dónde se usa	Características importantes
C	Termina el bucle o <code>switch</code> más cercano	<code>for</code> , <code>while</code> , <code>do-while</code> , <code>switch</code>	- Sale solo de la estructura más interna - No permite etiquetas
PHP	Termina el bucle o <code>switch</code>	<code>for</code> , <code>while</code> , <code>foreach</code> , <code>switch</code>	- Permite indicar niveles: <code>break 2;</code> -Sale de múltiples estructuras anidadas
JavaScript	Termina el bucle o <code>switch</code>	<code>for</code> , <code>while</code> , <code>do...while</code> , <code>switch</code>	- Puede usar etiquetas (<code>label:</code>) - <code>break label;</code> permite salir de bloques específicos anidados usando un numero -Evita ejecución de casos en <code>switch</code>
Ruby	Termina el bloque iterativo actual	<code>while</code> , <code>until</code> , iteradores (<code>each</code> , <code>loop</code>)	- Devuelve un valor opcional (<code>break valor</code>) al salir del ciclo

Ejercicio 9: Defina el concepto de ligadura y su importancia respecto de la semántica de un programa. ¿Qué diferencias hay entre ligadura estática y

dinámica? Cite ejemplos (proponer casos sencillos)

-Ligadura:

-Es el momento en el que a un atributo de una entidad se le asocia un valor determinado. Es importante ya que permite al compilador o interprete conocer el tipo y la ubicación de las variables y funciones, ofreciendo un correcto funcionamiento del programa

-Tipos de ligadura:

-Ligadura estática:

-Se establece antes de la ejecución. No se puede modificar

El termino estatica referencia al binding time y a su estabilidad (1 y 2)

-Ligadura dinamica:

-Se establece durante la ejecución. Se puede modificar durante la ejecución de acuerdo a alguna regla específica del lenguaje

-Excepcion: constantes (el binding es en ejecución/runtime pero no puede ser modificado luego de ser establecido)

-Momentos de ligadura (ejemplos):

-Estatico:

MOMENTO DE LIGADURA Y ESTABILIDAD	Ejemplo en lenguaje C
<u>En Definición del lenguaje</u> • La Forma de las sentencias • La Estructura del programa • Los Nombres de los tipos predefinidos	int Define los tipos permitidos, como se escriben, nombran, y lo vincula a operaciones algebraicas Int
<u>En Implementación</u> • Set de valores y su representación numérica • sus operaciones	- Vincula un tipo de variable a su representación en memoria, y determina el set de valores que están contenidos en el tipo. int a
<u>En Compilación</u> • Asignación/redefinición del tipo a las variables	- Liga tipo a la variable(atributo) - cambia el tipo en compilación si está permitido (ej. Caso Pascal)

-Dinamico:

- En Ejecución

- Variables se **enlazan** con sus **valores**
- Variables se **enlazan** con su **lugar** de **almacenamiento**

int a

a=10

a=15

- El **valor** de una variable entera se **liga en ejecución**.
- **puede cambiarse** muchas veces.